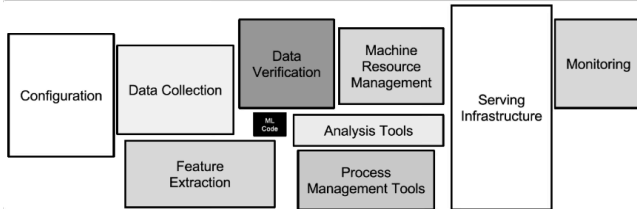


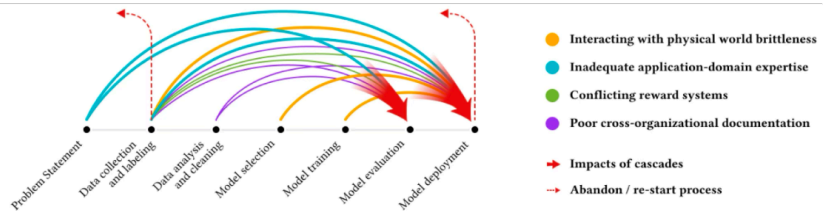
AI Infrastructure → Agent Harness

Arun Kejariwal

More than a decade ago, a NeurIPS [paper](#) argued that ML code accounts for only a small fraction of a real-world ML system, as illustrated by the small black box in the left-hand figure below. In a similar vein, a 2021 [study](#) exposed an imbalance in engineering attention: practitioners favored model development while undervaluing data work, allowing data-quality problems to compound into downstream failures, as shown in the right-hand figure below.



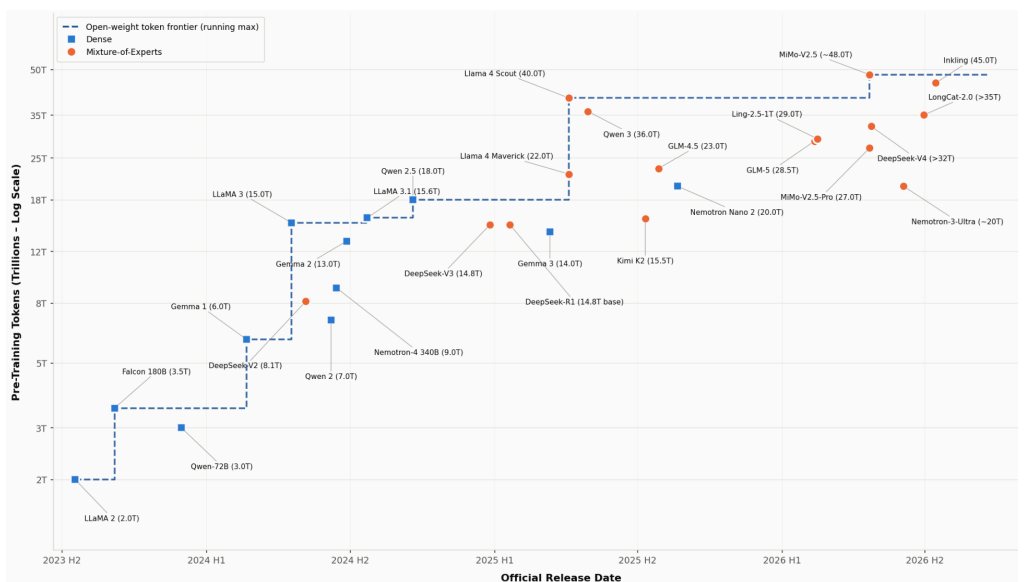
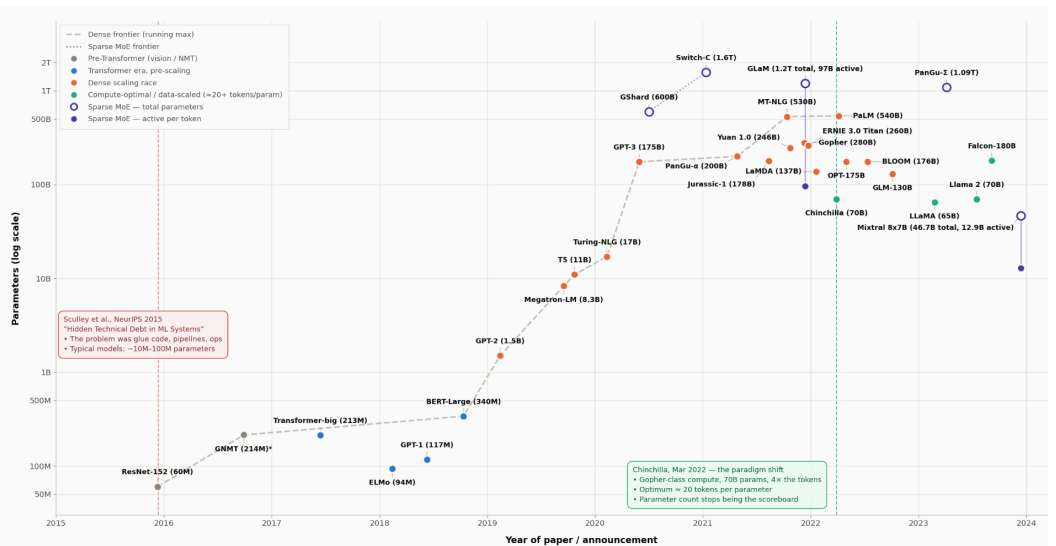
Source: "Hidden Technical Debt in Machine Learning Systems", 2015



Source: "Everyone wants to do the model work, not the data work". Data Cascades in High-Stakes AI, 2021

Yet the emphasis on increasing model parameter counts continued for several years, as showcased by the timeline on the right (note that the y-axis is in log scale). I saw this firsthand at Meta. Initially, larger deep learning models did help boost revenue and engagement across the different surfaces. After those early gains, further improvements increasingly came from scaling the data and the "data work" mentioned earlier. The former involved representing a wide variety of data through embeddings,¹ whose growing number and dimensionality increased memory requirements. The latter entailed reliable ingestion, validation, and preprocessing to keep accelerators fed.

A similar systems challenge is emerging with agents: realizing a model's capabilities depends on the harness - the runtime software that assembles context, manages state, coordinates tools and actions, and verifies outcomes. Here too, early attention centered on building larger models, including LLMs, VLMs, VLAs, and



¹ Embeddings are numerical vector representations of data. Beyond categorical features, they can encode numerical features and modalities such as text, images, audio, depth, thermal, and motion-sensor data.

VRMs. Subsequently, attention expanded from parameter counts to the quantity and quality of training data, as illustrated by the timeline above² (note that the y-axis is in log scale). The emphasis on scaling pretraining has also come under scrutiny. In October 2025, Andrej Karpathy [remarked](#) that “pre-training is not where you want to put most of your flops or your cost,” adding, “That’s why the models have gotten smaller.” He described the corresponding investment in mid-training and reinforcement learning. These observations motivate a related engineering question for agents: how much more performance can we extract by improving the system around an existing model?

The high cost of developing and operating advanced models strengthens the case for harness engineering: systematically optimizing the harness to improve agent performance without changing the underlying model.³ The parallel with AI infrastructure is direct: just as training infrastructure coordinates data and computation to produce a useful model, the harness coordinates information and actions to turn that model’s capabilities into reliable task completion. Recent results from harness optimization (keeping the underlying model fixed) include:

- Mean held-out lift of +14.2 pp ([BFCL multi-round](#)), +14.9 pp ([I²-Retail](#)), +10.1 pp ([I²-Telecom](#)) [1].
- Relative pass-rate gains of up to 132% across nine model-benchmark combinations [2].
- 7.6-13.6% relative improvement in average held-out pass rate [3].
- 12% fewer tokens than the initial harness on SWE-bench Verified, with higher aggregate success [4].

With open-weight models and routing platforms - such as [OpenRouter](#) and [Braintrust Gateway](#) - gaining traction, harness optimization is critical to bridge the performance gap with the (expensive) frontier models. As in [5], a harness can be modeled as a pair ($\mathcal{M}$, $\mathcal{C}$), where $\mathcal{M}$ is a model configuration and $\mathcal{C}$ is a harness configuration. $\mathcal{M}$ records the model→role (main, judge, evaluator) mapping and the fallback policy for each role; $\mathcal{C}$ records the agent’s behavior. The authors characterized the harness design space across nine categories (these are not independent dimensions), as captured by the Kiviatic plot below.



The plot uses paper counts⁴ as a proxy for research focus in academia and industry, showing uneven coverage across the nine categories. Beyond this corpus, safety - particularly model alignment⁵ - has been in the spotlight following recent incidents [6, 7, 8, 9, 10]. [Eval design](#) has also attracted considerable attention.

To make this design space actionable, the table separates control from safety, and treats planning and task decomposition as a distinct category, expanding the nine categories to 11. It then breaks each category into concrete subcategories and identifies what can be optimized. The objective is to improve verified task

² Models like Gemini, GPT, and Claude do not feature in the timeline owing to lack of official developer disclosure.
³ Refer to the [References](#) section for a snapshot of blogs and papers on harness engineering.
⁴ The corpus comprises 40 research papers and engineering reports (Sept’25 - Sept’26), with 16 industry-only, 5 academia-only, and 17 joint affiliations (two with unspecified affiliations are excluded). Note that a paper was counted towards an axis when it materially contributed to or studied that mechanism; merely using that component does not qualify.
⁵ Model alignment is a specific technical subfield and method within safety focused on ensuring an AI’s internal objectives, values, and behaviors accurately match human intentions and ethical principles.

completion within explicit cost and latency budgets and under required safety constraints. Changes should be validated on held-out tasks and checked for regressions.

Category	Subcategory	What to optimize?
Model selection	Role Binding	Which model serves each role - <code>main</code> , <code>judge</code> , <code>evaluator</code> - and its decoding settings.
	Routing & Fallback	Routing signals - cost, latency, difficulty - and thresholds; fallback model order and maximum attempts [11].
Context assembly	System Instructions	Instruction wording, prompt length, example selection, and example count [12].
	Conversation History	# retained turns; dropping vs. summarizing older turns; summary token limit [13].
	Evidence	# retrieved items shown and maximum tokens per item.
Memory management	Admission	Which facts, preferences, and task outcomes to store; writing during execution versus in the background [14].
	Retrieval	# memory items retrieved per step and matching method: keyword or embedding.
	Consolidation & Retention	Memory age that triggers summarization and maximum retention period.
	Provenance	Which metadata fields to retain: source, originating step, timestamp, and confidence.
	Shared State	Keys each agent may read/write; update rule - overwrite, append, or custom merge [15].
Tool ecosystem	Catalogue	Which tools are visible per task and how many are exposed simultaneously [16].
	Action Realization	Call format - JSON, tagged text, or code - and malformed-call handling: reject, retry, or repair [17].
	Result Handling	Maximum tokens from each tool result passed to the model.
	Skills	Which skills to load; startup versus on-demand loading of instructions and supporting files [18].
Execution environment	Sandbox Backend	Sandbox type - container or VM - and idle timeout in minutes before stopping [19].
	Isolation Scope	Which paths are readable or writable, which network domains are allowed, and fresh versus reused state between tasks [20, 21].
	Provisioning	CPU cores, memory allocation, runtime limit, and # concurrent rollouts.
	Credential Exposure	Which secrets each tool can access; loading at startup versus fetching per call.
Evaluation and reward	Verifiers	Mix of code-based, model-based, and human checks; # checks and required agreement [22].
	Reward Definition	Reward components, their weights, and thresholds for task success.
	Metric Sensitivity	Binary versus graded scoring; number of test tasks and repeats; minimum detectable score difference [23].
	Integrity	# human-scored calibration cases; answer-order swapping for pairwise judges; percentage of scores independently rechecked [22, 24].
Control and orchestration	Single-Agent Flow	Maximum steps, stopping conditions, and retry count per failure.

	Multi-Agent Orchestration	# workers, sequential versus parallel execution, coordination topology, and maximum delegation depth [25].
	Human-in-the-loop	Which actions require approval, permitted responses - approve, edit, reject - and approval timeout [26].
	Budgets	Maximum tokens, calls, steps, and elapsed time; action taken when each limit is reached.
Observability	Event Capture	Which event types to record and what percentage to sample under load.
	Event Detail	Which input/output fields to retain and maximum bytes or tokens recorded per call.
	Causal Linking	Which operations receive separate trace records and how parent-child links connect model and tool calls [27].
	Checkpoints	Which state fields to persist, checkpoint frequency, and # checkpoints retained [28].
	Cost & Regression	Which cost and latency counters to track; absolute or percentage thresholds for flagging regressions.
Training bridge	Trace Selection	For group-relative RL: retaining versus dropping uniform-reward groups; including versus masking truncated traces in the training loss [29].
	Buffer Lifecycle	Maximum # buffered traces and eviction order, such as oldest first.
	Training Metadata	Which annotations to retain, depending on the RL algorithm: rewards, step boundaries, generating-model log-probabilities, and model/harness identifiers [30].
	Grouping	# traces per task group and which model/harness versions may be pooled for group-relative RL.
Safety and governance	Capability Enforcement	Which tools and paths each role may access; default allow versus deny.
	Adversarial Surface	Which external inputs to filter, validation rules, and blocking versus human review when checks fail [31].
	Audit & Rollback	Audit events and retention period; tamper-detection checks and read-only storage; # restorable harness-state versions [32, 33].
Planning and task decomposition	Planner Strategy	Planning scheme - reason-act [34], plan-then-execute [35], or search [36] - and # candidate branches explored.
	Decomposition	Maximum decomposition depth and # subtasks per plan.
	Plan Revision	When to replan; patching failed steps versus regenerating the plan; # completed steps retained.
	Plan Representation	Plan format - prose, checklist, task graph - and which parts the agent may edit.

The lesson from AI infrastructure is that gains depend on how well the entire system turns resources into useful capability. Agent harnesses bring that same engineering challenge to the runtime: supplying the right context, coordinating tools and actions, verifying outcomes, and turning experience into improvements. For open-weight models, this creates a concrete opportunity to close the gap with proprietary frontier models on specific tasks. The table above outlines an engineering agenda: diagnose failures, optimize the responsible mechanisms, and verify gains against quality, cost, and latency constraints. Harness optimization deserves the same rigor that made AI infrastructure a source of competitive advantage.

References

- Blogs
 - [Aug'26] [Knowing When to Stop: The Art of Making a Loop Converge](#)
 - [Aug'26] [FinanceHarness: Autonomous Financial Deep Research Framework](#)
 - [Aug'26] [A Domain-Specific Harness for End-to-End Automation of Optimization Research](#)
 - [June'26] [A Field Guide to Agent Harness Engineering](#)
 - [May'26] [The Agent Harness: Why the LLM Is the Smallest Part of Your Agent System](#)
 - [March'26] [Harness design for long-running application development](#)
 - [March'26] [The Anatomy of an Agent Harness](#)
 - [March'26] [Skill Issue: Harness Engineering for Coding Agents](#)
 - [Feb'26] [Harness engineering: leveraging Codex in an agent-first world](#)
 - [Nov'25] [Effective harnesses for long-running agents](#)
- Papers
 - [Aug'26] [Self-Harness: Harnesses That Improve Themselves](#)
 - [Aug'26] [Context-to-Harness Skill Compilation for Self-Evolving Agents](#)
 - [Aug'26] [Verify Smarter, Evolve Further: Efficient Harness Evolution through Behavior-Aware Verification](#)
 - [July'26] [Harness VLA: Steering Frozen VLAs into Reliable Manipulation Primitives via Memory-Guided Agents](#)
 - [July'26] [Recursive Harness Self-Improvement](#)
 - [June'26] [From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws](#)
 - [April'26] [Observability-Driven Automatic Evolution of Coding-Agent Harnesses](#)
 - [March'26] [Meta-Harness: End-to-End Optimization of Model Harnesses](#)
 - [March'26] [Building AI Coding Agents for the Terminal](#)